บทที่ 7: ธุรกรรม (Transactions)

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 20–22 นาที)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่เจ็ด ว่าด้วยเรื่อง ธุรกรรม หรือ Transactions

ในระบบฐานข้อมูล มีสิ่งแย่ ๆ หลายอย่างที่อาจเกิดขึ้นได้ตลอดเวลา เช่น

- **ฐานข้อมูลอาจดับกลางคันระหว่างกำลังเขียนข้อมูล**
- **แอปพลิเคชันอาจพังค้าง หรือเครือข่ายอาจขาดตอนไปเฉย ๆ**
- **มีผู้ใช้หลายคนพยายามแก้ไขข้อมูลชุดเดียวกันพร้อมกัน ทำให้ข้อมูลตีกันเละเทะ**
- **ดิสก์เต็มกะทันหันขณะกำลังบันทึก**

ถ้าไม่มีเครื่องมือป้องกัน ข้อมูลคงเสียหายและใช้งานจริงไม่ได้ตั้งนานแล้ว แนวคิดที่เข้ามาช่วยแก้ปัญหาเหล่านี้ ตั้งแต่ยุคแรกของฐานข้อมูลคือ ธุรกรรม (Transaction)

หน้าที่ของธุรกรรมคือ รวบรวมการอ่านและการเขียนหลาย ๆ คำสั่งเข้าเป็นกลุ่มก้อนเดียว แล้วรับประกันตาม หลักการที่เรียกว่า ACID ถ้ามีอะไรผิดพลาดแม้แต่นิดเดียว ระบบจะยกเลิกการทำงานทั้งหมดเสมือนว่าไม่มี อะไรเกิดขึ้น

ฟังดูเหมือนเป็นตัวช่วยที่วิเศษมาก แต่ในระบบสมัยใหม่ โดยเฉพาะระบบแบบกระจาย ธุรกรรมก็มีราคาที่ต้อง จ่าย ทั้งเรื่องความเร็วและความซับซ้อน บทนี้จะพาไปดูว่าเบื้องหลังของ ACID คืออะไร และแต่ละระดับการ ป้องกันทำงานอย่างไรครับ

ตอนที่ 2: ความหมายของคำว่า ACID

คำว่า ACID ถูกบัญญัติขึ้นในปี 1983 เพื่อสร้างมาตรฐานการค้าของฐานข้อมูลเชิงสัมพันธ์ ประกอบด้วย 4 ตัวอักษร แต่ในทางปฏิบัติ แต่ละระบบตีความคำเหล่านี้ต่างกันพอสมควร

ตัวแรก A ย่อมาจาก Atomicity (ความเป็นปรมาณู) คนมักสบสนคำว่า Atomicity กับ Concurrency แต่ ความจริงในบริบทนี้ มันแปลว่า "ทั้งหมดหรือไม่มีเลย" (all-or-nothing) ถ้าธุรกรรมมี 5 ขั้นตอน ทำไปได้ 3 ขั้นตอนแล้วระบบดับ ฐานข้อมูลต้องล้างข้อมูลของ 2 ขั้นตอนแรกทิ้งไปทั้งหมด เพื่อไม่ให้ข้อมูลค้างอยู่ใน สภาพครึ่ง ๆ กลาง ๆ

ตัวที่สอง C ย่อมาจาก Consistency (ความสอดคล้อง) คำนี้มีความหมายกว้างที่สุด ใน ACID คำว่า Consistency หมายถึง "กฎระเบียบของข้อมูลที่เป็นจริงเสมอ" เช่น ยอดเงินรวมในบัญชีต้องเป็นศูนย์ หรือ บัญชีห้ามติดลบ การรักษากฎเหล่านี้ส่วนใหญ่เป็นหน้าที่ของแอปพลิเคชันที่ต้องส่งข้อมูลที่ถูกต้องมาให้ ฐานข้อมูล ฐานข้อมูลเป็นแค่ผู้ช่วยตรวจสอบตามกฎที่ตั้งไว้

ตัวที่สาม **I** ย่อมาจาก **Isolation (**ความเป็นอิสระ) หมายความว่า ธุรกรรมหลายตัวที่ทำงานพร้อมกัน จะต้องมองไม่เห็นการเปลี่ยนแปลงของกันและกัน เสมือนว่าแต่ละตัวรันอยู่คนเดียวบนโลก เพื่อป้องกันไม่ให้ข้อมูลตีกัน แต่ในความเป็นจริง การบังคับให้ทำทีละตัวทำให้ระบบช้ามาก ฐานข้อมูลส่วนใหญ่จึงยอมผ่อนปรนเรื่องนี้ เรียกว่ามีระดับการแยก (**Isolation Levels)** ที่ต่างกัน

ตัวที่สี่ **D** ย่อมาจาก **Durability (**ความคงทน) หมายความว่า เมื่อธุรกรรมบอกว่าสำเร็จแล้ว ข้อมูลจะต้องไม่หายไปไหน แม้ไฟจะดับหรือเครื่องจะพังทันที ในฐานข้อมูลเดี่ยว แปลว่าข้อมูลถูกเขียนลงดิสก์เรียบร้อยแล้ว (ผ่าน **Write-Ahead Log** หรือเครื่องมืออื่น) ส่วนในระบบกระจาย แปลว่าข้อมูลถูกคัดลอกไปยังสำเนาหลายเครื่องแล้ว

ตอนที่ **3:** ความปลอดภัยจากการทำงานพร้อมกัน (**Concurrency Control)**

ถ้ามีผู้ใช้คนเดียวใช้งาน ฐานข้อมูลคงไม่มีปัญหาเรื่องข้อมูลตีกัน แต่ในความเป็นจริงมีผู้ใช้เป็นล้านคนเข้ามาพร้อมกัน

หากฐานข้อมูลปล่อยให้ใครอยากแก้อะไรก็แก้พร้อมกัน จะเกิดปัญหาที่เรียกว่า **Race Condition** ซึ่งนำไปสู่ความผิดพลาดหลายรูปแบบ ระบบจึงต้องมีกลไกควบคุม

ปัญหาการอ่านข้อมูลที่ยังไม่สมบูรณ์ (**Dirty Read)** สมมติธุรกรรม **A** กำลังแก้ไขยอดเงิน แต่ยังไม่กดบันทึก (ยังไม่ **Commit)** ระหว่างนั้นธุรกรรม **B** เข้ามาอ่านยอดเงินนั้นไปคำนวณต่อ ปรากฏว่าภายหลังธุรกรรม **A** เกิดทำพลาดแล้วยกเลิก (**Abort)** ยอดเงินที่ธุรกรรม **B** อ่านไปจึงกลายเป็นข้อมูลปลอมที่ไม่มีอยู่จริง

ปัญหาการอ่านซ้ำแล้วได้ค่าไม่เหมือนกัน (**Non-repeatable Read** หรือ **Read Skew)** สมมติผู้ใช้มีเงินอยู่ในบัญชีสองบัญชี บัญชีละ **500** บาท รวมเป็น **1,000** บาท ธุรกรรม **A** กำลังโอนเงิน **100** บาทจากบัญชีแรกไปบัญชีที่สอง ระหว่างที่ธุรกรรม **A** เพิ่งหักเงินจากบัญชีแรกเสร็จแต่ยังไม่ได้เติมเข้าบัญชีที่สอง ผู้ใช้เปิดดูยอดเงินพอดี ปรากฏว่าเห็นบัญชีแรกมี **400** บัญชีที่สองมี **500** ยอดรวมเหลือแค่ **900** บาท ทั้งที่เงินไม่ได้หายไปไหน แค่อ่านเจอจังหวะที่ระบบยังทำไม่เสร็จ

ปัญหาการเขียนทับซ้อนกัน (**Lost Update)** สมมติพนักงานสองคนอ่านข้อมูลหน้าเดียวกันพร้อมกัน เห็นตัวเลขเป็น **10** คนแรกบวกเพิ่ม **1** คนที่สองบวกเพิ่ม **1** แล้วกดบันทึกพร้อมกัน คนที่บันทึกทีหลังจะเขียนทับค่าของคนแรก ทำให้ยอดที่ควรจะเพิ่มเป็น **12** เหลือแค่ **11** เพราะข้อมูลของคนแรกถูกลบหายไป

ปัญหาการอ่านเจอข้อมูลใหม่ที่เพิ่มขึ้นระหว่างทาง (**Write Skew** และ **Phantom Reads)** เป็นปัญหาที่ซับซ้อนขึ้น เช่น กฎของโรงพยาบาลบอกว่าต้องมีเวรแพทย์อยู่ประจำอย่างน้อย **2** คน หมอสองคนเห็นว่ามีคนอยู่ **2** คนพอดี เลยกดขอลาหยุดพร้อมกันในเวลาไล่เลี่ยกัน ทั้งคู่ต่างตรวจสอบแล้วพบว่ายังทำได้เพราะยังมีคนอื่นเหลืออยู่ แต่พอระบบบันทึกทั้งสองรายการพร้อมกัน กลายเป็นไม่มีแพทย์อยู่เวรเลยสักคน

ตอนที่ **4:** ระดับการแยกธุรกรรม (**Isolation Levels)**

เพื่อแลกกับประสิทธิภาพ ฐานข้อมูลจึงเปิดให้เลือกความเข้มงวดในการป้องกันปัญหาเหล่านี้ แบ่งเป็นระดับต่าง ๆ ดังนี้ครับ

1. ระดับ **Read Committed** เป็นระดับเริ่มต้นของฐานข้อมูลส่วนใหญ่ ป้องกันได้ **2** เรื่อง คือ

- ป้องกัน **Dirty Read:** จะไม่มีใครอ่านข้อมูลที่ยังไม่ **Commit** เด็ดขาด
- ป้องกัน **Dirty Write:** จะไม่มีใครเขียนทับข้อมูลที่กำลังรอ **Commit** อยู่

แต่ระดับนี้ยังปล่อยให้เกิด **Non-repeatable Read** ได้ คือถ้าอ่านรอบแรกกับรอบสองห่างกัน แล้วมีคนอื่นแอบมาแก้ข้อมูลตรงกลาง เราจะเห็นค่าที่ไม่เหมือนกัน

เบื้องหลังการทำงาน: ส่วนใหญ่ใช้เทคนิคการจำค่าเก่าไว้ (**Row-level lock** ตอนเขียน และเก็บค่าเก่าไว้ให้คนอ่านอ่านค่าเดิมก่อนแก้เสร็จ)

2. ระดับ **Snapshot Isolation (Repeatable Read)** เป็นระดับที่นิยมมาก ป้องกัน **Non-repeatable Read** ได้อย่างเด็ดขาด

หลักการคือ เมื่อธุรกรรมเริ่มขึ้น ฐานข้อมูลจะถ่ายภาพข้อมูล (**Snapshot)** ณ เวลานั้นไว้ให้ธุรกรรมนั้นใช้มองโลกใบส่วนตัวตลอดรอดฝัน แม้จะมีธุรกรรมอื่นมาแก้ข้อมูลและบันทึกเสร็จไปแล้ว ธุรกรรมของเราก็จะไม่เห็นการเปลี่ยนแปลงนั้น จะเห็นแต่โลกเดิมตอนที่ตัวเองเริ่มต้น

เบื้องหลังการทำงาน: เรียกว่า **MVCC (Multi-Version Concurrency Control)** ฐานข้อมูลจะไม่เขียนทับข้อมูลเดิมตรง ๆ แต่จะสร้างเวอร์ชันใหม่แปะป้ายหมายเลขกำกับไว้ เมื่อใครต้องการอ่านข้อมูลเวอร์ชันไหน ระบบก็จะเลือกเวอร์ชันที่ตรงกับเวลาเริ่มต้นของธุรกรรมนั้นให้ เทคนิคนี้ทำให้คนอ่านไม่ต้องรอคนเขียน และคนเขียนก็ไม่ต้องรอคนอ่าน ทำงานพร้อมกันได้อย่างราบรื่น

3. การป้องกันความเสียหายระดับสูงสุด: **Serializability** แม้จะมี **Snapshot Isolation** แต่ก็ยังมีช่องโหว่บางอย่าง เช่น ปัญหาหมอลาหยุดพร้อมกัน (**Write Skew)** หรือปัญหาการสร้างข้อมูลใหม่ทับซ้อนกัน (**Phantom Read)**

ระดับสูงสุดที่ปลอดภัยไร้ที่ติคือ **Serializable** หมายความว่า ฐานข้อมูลจะรับประกันว่า ผลลัพธ์สุดท้ายของการรันธุรกรรมหลายตัวพร้อมกัน จะต้องออกมาถูกต้องเหมือนกับการรันทีละตัวต่อแถวกันทีละอันเป๊ะ ๆ ไม่มีผิดเพี้ยน

ตอนที่ **5:** วิธีการบรรลุระดับ **Serializable**

การจะทำให้ระบบเป็น **Serializable** มีอยู่ **3** แนวทางหลักที่นิยมใช้กันในโลกของฐานข้อมูล

วิธีที่ **1:** การรันทีละคำสั่งจริง ๆ (**Actual Serial Execution)** แนวทางนี้ง่ายที่สุด คือ เลิกให้ซีพียูทำหลายอย่างพร้อมกัน ให้มีเธรดเดียว (**Single-threaded)** ประมวลผลธุรกรรมทีละอันต่อแถวกันไปเลย

ฟังดูเหมือนจะช้า แต่ถ้าข้อมูลทั้งหมดใส่ไว้ในหน่วยความจำ (**In-memory database)** และแต่ละธุรกรรมใช้เวลาสั้นมาก ระบบจะทำความเร็วได้สูงมากโดยไม่ต้องเสียเวลามาคอยล็อกข้อมูลให้ยุ่งยาก ฐานข้อมูลอย่าง **Redis** หรือ **VoltDB** ใช้แนวทางนี้

ข้อจำกัดคือ ปริมาณงานทั้งหมดต้องจำกัดอยู่แค่ซีพียูหัวเดียว และถ้ามีธุรกรรมไหนรันนาน งานทุกอย่างข้างหลังจะติดค้างรอหมดทันที

วิธีที่ 2: การล็อกแบบสองขั้นตอน (**Two-Phase Locking - 2PL**) เป็นวิธีมาตรฐานของฐานข้อมูลเชิงสัมพันธ์แบบดั้งเดิมที่ใช้กันมาหลายสิบปี

หลักการคือ

- ถ้าใครจะอ่านข้อมูล ต้องขอ **Shared Lock** คนอื่นอ่านด้วยได้ แต่ห้ามแก้
- ถ้าใครจะแก้ข้อมูล ต้องขอ **Exclusive Lock** ห้ามคนอื่นยุ่งเด็ดขาดจนกว่าจะทำเสร็จ
- ล็อกจะถูกถือครองไว้จนกระทั่งจบธุรกรรม (**Commit** หรือ **Abort**) ถึงจะปลดล็อกทีเดียว

วิธีนี้ป้องกันได้ทุกปัญหา แต่มีราคาที่ต้องจ่ายสูงมาก คือ ความช้าและการติดล็อกตายหมู่ (**Deadlock**) เช่น ธุรกรรม **A** ถือล็อกชิ้นที่ **1** แล้วรอขอล็อกชิ้นที่ **2** ในขณะที่ธุรกรรม **B** ถือล็อกชิ้นที่ **2** แล้วมารอขอล็อกชิ้นที่ **1** ทั้งคู่จะยืนจ้องตากันไปไหนไม่ได้ ระบบต้องคอยตรวจจับแล้วสั่งฆ่า (**Abort**) ฝ่ายใดฝ่ายหนึ่งทิ้งเพื่อให้รอด

วิธีที่ 3: แบบ **optimistic concurrency control (SSI - Serializable Snapshot Isolation)** วิธีสมัยใหม่ที่ไม่ชอบการล็อกตั้งแต่แรก เพราะคิดว่าโอกาสชนกันมีน้อย

หลักการคือ ปล่อยให้ทุกอย่างรันไปตามสบายด้วย **Snapshot Isolation** แต่ระบบจะคอยแอบจดบันทึกไว้ว่าธุรกรรมไหนอ่านข้อมูลอะไรไปบ้าง

ก่อนที่ธุรกรรมจะกดบันทึก (**Commit**) ระบบจะตรวจสอบย้อนหลังว่า ข้อมูลที่เคยอ่านไปตอนต้น ตอนนี้ถูกคนอื่นแอบเปลี่ยนไปหรือยัง ถ้าเปลี่ยนไปแล้ว แปลว่ามีความเสี่ยงที่จะเกิดความผิดพลาด ระบบจะไม่อนุญาตให้บันทึก แต่จะสั่งยกเลิก (**Abort**) แล้วให้แอปพลิเคชันลองทำใหม่อีกรอบ

ข้อดีคือ ไม่มีการล็อกรอ ทำให้ระบบรวดเร็ว แต่ถ้างานไหนมีคนแย่งกันแก้อย่างดุเดือด (**Contention** สูง) ธุรกรรมจะถูกสั่ง **Abort** ซ้ำซากจนระบบแทบไม่เดินหน้า ซึ่งต้องระวัง

ตอนที่ 6: สรุปท้ายบท

มาทบทวนประเด็นสำคัญของบทนี้กันครับ

หนึ่ง ธุรกรรมคือเครื่องมือสำคัญที่ช่วยให้การจัดการข้อมูลปลอดภัย โดยยึดหลัก **ACID** เพื่อป้องกันความเสียหายเมื่อเกิดเหตุการณ์ไม่คาดฝัน

สอง ระบบต้องเผชิญกับปัญหาข้อมูลตีกันจากการทำงานพร้อมกัน เช่น **Dirty Read, Non-repeatable Read** และ **Lost Update**

สาม ระดับการแยกธุรกรรม (**Isolation Levels**) มีตั้งแต่ **Read Committed** ที่เร็วแต่มีความเสี่ยง, **Snapshot Isolation (MVCC)** ที่นิยมใช้เพราะอ่านเขียนลื่นไหลโดยไม่แย่งกัน, จนถึงระดับสูงสุดคือ **Serializable** ที่ปลอดภัยไร้ที่ติ

สี่ การไปถึงระดับ **Serializable** ทำได้ **3** ทาง คือ การรันทีละคำสั่งในหน่วยความจำ, การล็อกแบบเข้มงวด (**2PL**), และการตรวจสอบย้อนหลังแบบใจดี (**SSI**)

บทต่อไปคือบทที่แปด เราจะข้ามกลับไปดูโลกของระบบกระจายอีกครั้ง ว่าด้วยเรื่องของความท้าทายในเครือข่ายและความน่าเชื่อถือของเวลา ขอบคุณที่ฟังครับ

ทิปสำหรับ **Edge:** บทนี้มีศัพท์เฉพาะทางเทคนิคค่อนข้างเยอะ เช่น **MVCC, 2PL, SSI** ถ้าฟังแล้วรู้สึกนามธรรมเกินไป ให้จำภาพง่าย ๆ ว่า "ระบบธุรกรรมเหมือนเวลาเราโอนเงินในแอปธนาคาร ที่ต้องมั่นใจว่าเงินไม่หายและไม่มีใครมาแอบแก้ตัวเลขกลางทางครับ"

จะให้ทำบทที่ **8** ต่อเลยไหมครับ หรือต้องการให้สรุปภาพรวมภาคที่ **2** เพิ่มเติมอย่างไรแจ้งได้เลยครับ 😊